

BadApp – Violations V1–V167

Violated code, rectified code, and concise explanation for each issue.

V1 — Star import (java.util.*)

Violated code:

```
import java.util.*;
```

Rectified code:

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.Hashtable;
import java.util.List;
import java.util.Map;
import java.util.HashMap;
import java.util.Random;
```

Explanation:

Avoid '*' imports; make dependencies explicit to prevent collisions.

V2 — Star import (java.io.*)

Violated code:

```
import java.io.*;
```

Rectified code:

```
import java.io.BufferedReader;
import java.io.FileInputStream;
import java.io.FileReader;
import java.io.IOException;
import java.io.InputStream;
```

Explanation:

Use explicit imports for clarity.

V3 — Star import (java.sql.*)

Violated code:

```
import java.sql.*;
```

Rectified code:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
```

Explanation:

Explicit imports; also prefer PreparedStatement over Statement.

V4 — Unused import (java.net.*)

Violated code:

```
import java.net.*;
```

Rectified code:

```
// removed
```

Explanation:

Remove unused imports to reduce warnings.

V5 — Unused star import (java.util.concurrent.*)

Violated code:

```
import java.util.concurrent.*;
```

Rectified code:

```
// removed
```

Explanation:

Remove unused imports.

V6 — Unused import (java.text.*)

Violated code:

```
import java.text.*;
```

Rectified code:

```
// removed
```

Explanation:

Remove unused imports.

V7 — Unused static import (Math.*)

Violated code:

```
import static java.lang.Math.*;
```

Rectified code:

```
// removed
```

Explanation:

Avoid unused static imports.

V8 — Reduce SSL wildcard import

Violated code:

```
import javax.net.ssl.*;
```

Rectified code:

```
import javax.net.ssl.SSLContext;
import javax.net.ssl.TrustManager;
import javax.net.ssl.X509TrustManager;
```

Explanation:

Keep only types that are used.

V9 — Default package (missing package declaration)

Violated code:

```
// (no package)
```

Rectified code:

```
package com.badcode;
```

Explanation:

Always declare a package.

V10 — God class / too many responsibilities

Violated code:

```
public class BadApp { /* giant main and many concerns */ }
```

Rectified code:

```
public final class App {  
    public static void main(String[] args){ new AppRunner().run(args); }  
}  
final class AppRunner { /* orchestrates */ }  
final class DbService { /* DB */ }  
final class FileService { /* IO */ }  
final class SecurityService { /* security */ }
```

Explanation:

Split by concerns to improve cohesion and testability.

V11 — Public mutable static field

Violated code:

```
public static List sharedList = new ArrayList();
```

Rectified code:

```
private static final List<Object> SHARED_LIST =  
    java.util.Collections.synchronizedList(new java.util.ArrayList<>());
```

Explanation:

Make it private/final, typed, and thread-safe.

V12 — Raw type (no generics)

Violated code:

```
public static List sharedList = new ArrayList();
```

Rectified code:

```
private static final List<Object> SHARED_LIST =  
    java.util.Collections.synchronizedList(new java.util.ArrayList<>());
```

Explanation:

Use generics for type safety.

V13 — Public mutable array exposed

Violated code:

```
public static String[] GLOBAL_DATA = new String[] { "a", "b", "c" };
```

Rectified code:

```
private static final String[] GLOBAL_DATA = { "a", "b", "c" };  
public static String[] getGlobalDataCopy(){  
    return java.util.Arrays.copyOf(GLOBAL_DATA, GLOBAL_DATA.length);  
}
```

Explanation:

Keep arrays private; return defensive copies.

V14 — Hardcoded credentials/secrets

Violated code:

```
public static final String PASSWORD = "P@ssw0rd!";  
public static final String API_KEY = "sk_live_123456";
```

Rectified code:

```
private static final String PASSWORD = System.getenv("APP_PASSWORD");
private static final String API_KEY = System.getenv("APP_API_KEY");
```

Explanation:

Use environment/secret store; never hardcode secrets.

V15 — Non-final static constant

Violated code:

```
public static String ENV = "prod";
```

Rectified code:

```
private static final String ENV = System.getenv().getOrDefault("APP_ENV", "prod");
```

Explanation:

Constants should be final; prefer env-driven config.

V16 — Mutable static counter not thread-safe

Violated code:

```
public static int counter = 0;
```

Rectified code:

```
private static final java.util.concurrent.atomic.AtomicInteger COUNTER =
    new java.util.concurrent.atomic.AtomicInteger(0);
```

Explanation:

Use AtomicInteger/LongAdder for thread-safe counters.

V17 — Magic number threshold

Violated code:

```
private static int threshold = 42;
```

Rectified code:

```
private static final int DEFAULT_THRESHOLD = 42;
```

Explanation:

Name important constants to convey intent.

V18 — Overly broad access modifier

Violated code:

```
public static Map config = new HashMap();
```

Rectified code:

```
private static final Map<String,Integer> CONFIG = new HashMap<>();
```

Explanation:

Use least privilege and generics.

V19 — Raw Map type

Violated code:

```
public static Map config = new HashMap();
```

Rectified code:

```
private static final Map<String,Integer> CONFIG = new HashMap<>();
```

Explanation:

Use generics for type safety.

V20 — Dead field never used

Violated code:

```
private static boolean FLAG = false;
```

Rectified code:

```
// removed
```

Explanation:

Remove unused fields.

V21 — Field shadowed later

Violated code:

```
private static String path = "file.txt";
```

Rectified code:

```
private static final String DEFAULT_PATH = "file.txt";
```

Explanation:

Use distinct, descriptive names to avoid shadowing.

V22 — Synchronized method with blocking IO

Violated code:

```
public static synchronized void printSlow(String s){ try{ Thread.sleep(50); System.out.println(s);}catch(Exception e){} }
```

Rectified code:

```
private static final java.util.logging.Logger LOG = java.util.logging.Logger.getLogger("App");  
public static void printMessage(String s){ LOG.info(s); }
```

Explanation:

Avoid synchronized + sleep around IO; use logger.

V23 — Sleep in synchronized block

Violated code:

```
// see V22
```

Rectified code:

```
// fixed by removing sync and sleep
```

Explanation:

Sleeping under lock hurts concurrency.

V24 — System.out instead of logger

Violated code:

```
System.out.println(s);
```

Rectified code:

```
LOG.info(s);
```

Explanation:

Prefer logging framework.

V25 — Swallowing generic Exception

Violated code:

```
catch(Exception e){}
```

Rectified code:

```
catch (InterruptedException ie){ Thread.currentThread().interrupt(); LOG.log(java.util.logging.Level.WARNING, "Interrupted", ie);}  
catch (java.io.IOException ioe){ LOG.log(java.util.logging.Level.SEVERE, "I/O error", ioe); }
```

Explanation:

Catch specific exceptions; log; restore interrupt.

V26 — Empty catch block

Violated code:

```
catch(Exception e){}
```

Rectified code:

```
catch(Exception e){ LOG.log(java.util.logging.Level.WARNING, "Unexpected error", e); }
```

Explanation:

Never swallow exceptions silently.

V27 — Ignoring command-line args sanity

Violated code:

```
public static void main(String[] args){ /* ... */ }
```

Rectified code:

```
public static void main(String[] args){ if(args==null) args=new String[0]; new AppRunner().run(args); }
```

Explanation:

Validate args and delegate to runner.

V28 — System.out in main

Violated code:

```
System.out.println("Starting BadApp");
```

Rectified code:

```
LOG.info("Starting App");
```

Explanation:

Use logger instead of stdout.

V29 — Overly long main

Violated code:

```
// giant main with many concerns
```

Rectified code:

```
public final class App { public static void main(String[] args){ new AppRunner().run(args); } }
```

Explanation:

Decompose responsibilities (see V10).

V30 — Duplicate code blocks

Violated code:

```
for(int i=0;i<3;i++){ counter++; }
```

Rectified code:

```
private static void incrementCounter(java.util.concurrent.atomic.AtomicInteger c,int times){ for(int i=0;i<times;i++){ c.incrementAndGet(); } }
```

Explanation:

Extract reusable method (DRY).

V31 — Mutable static without synchronization

Violated code:

`counter++;`

Rectified code:

`COUNTER.incrementAndGet();`

Explanation:

Use AtomicInteger.

V32 — Unused local (a)

Violated code:

`int a = 1;`

Rectified code:

`// removed`

Explanation:

Remove unused locals.

V33 — Unused local (b)

Violated code:

`int b = 2;`

Rectified code:

`// removed`

Explanation:

Remove unused locals.

V34 — Unused local (c)

Violated code:

`int c = 3;`

Rectified code:

`// removed`

Explanation:

Remove unused locals.

V35 — Unused local (d)

Violated code:

`int d = 4;`

Rectified code:

`// removed`

Explanation:

Remove unused locals.

V36 — Unused local (e)

Violated code:

`int e = 5;`

Rectified code:

`// removed`

Explanation:

Remove unused locals.

V37 — Unused locals (group)

Violated code:

```
// many
```

Rectified code:

```
// removed
```

Explanation:

Remove unused locals.

V38 — Unused string (s1)

Violated code:

```
String s1 = "hello";
```

Rectified code:

```
// removed
```

Explanation:

Remove or use meaningfully.

V39 — Unused string (s2)

Violated code:

```
String s2 = "world";
```

Rectified code:

```
// removed
```

Explanation:

Remove or use meaningfully.

V40 — Magic number for π

Violated code:

```
double piApprox = 3.14;
```

Rectified code:

```
double piApprox = Math.PI;
```

Explanation:

Use Math.PI or a named constant.

V41 — Compare string with ==

Violated code:

```
if(s1 == "hello"){ /*...*/ }
```

Rectified code:

```
if("hello".equals(s1)){ /*...*/ }
```

Explanation:

Use equals for value equality.

V42 — System.out for diagnostics

Violated code:

```
System.out.println("Bad compare");
```

Rectified code:

```
LOG.warning("Bad compare");
```

Explanation:

Use logger at appropriate level.

V43 — Unnecessary boxing / deprecated ctor

Violated code:

```
Integer boxed = new Integer(10); int unboxed = boxed;
```

Rectified code:

```
Integer boxed = Integer.valueOf(10); int unboxed = boxed.intValue();
```

Explanation:

Avoid deprecated constructors; rely on valueOf/autoboxing.

V44 — Null dereference risk

Violated code:

```
String maybeNull = null; maybeNull.length();
```

Rectified code:

```
String v = getValue(); if(v != null){ LOG.info(String.valueOf(v.length())); }
```

Explanation:

Null-check or use Optional.

V45 — Redundant if/else

Violated code:

```
if(true){} else {}
```

Rectified code:

```
// removed
```

Explanation:

Remove unreachable/no-op branches.

V46 — Empty try/finally

Violated code:

```
try{} finally{}
```

Rectified code:

```
// removed
```

Explanation:

Remove no-op blocks.

V47 — Overly broad catch with ignore

Violated code:

```
try{ riskyMethod(); } catch(Throwable t){}
```

Rectified code:

```
try{ safeMethod(); } catch(java.sql.SQLException e){ LOG.log(java.util.logging.Level.SEVERE, "DB error", e); }
```

Explanation:

Catch only what you can handle; not Throwable.

V48 — SQL injection in callee

Violated code:

```
riskyMethod("name", "DROP TABLE users;");
```

Rectified code:

```
safeUpdate("name", "UPDATE users SET active=? WHERE name=?");
```

Explanation:

Use parameterized SQL; never execute user-provided SQL.

V49 — Catching Throwable

Violated code:

```
catch(Throwable t){}
```

Rectified code:

```
catch(RuntimeException e){ LOG.log(java.util.logging.Level.WARNING, "Unexpected", e); }
```

Explanation:

Don't catch Throwable (includes Errors).

V50 — Empty catch

Violated code:

```
catch(Exception e){}
```

Rectified code:

```
catch(Exception e){ LOG.log(java.util.logging.Level.WARNING, "Suppressed", e); }
```

Explanation:

Always log/handle exceptions.

V51 — Deprecated Thread.stop()

Violated code:

```
Thread t = new Thread(new BadRunnable()); t.stop();
```

Rectified code:

```
Thread t = new Thread(new GoodRunnable()); t.start(); t.interrupt();
```

Explanation:

Use cooperative interruption; never stop().

V52 — Calling run() directly

Violated code:

```
t.run();
```

Rectified code:

```
t.start();
```

Explanation:

start() spawns a new thread; run() is synchronous.

V53 — Unchecked cast

Violated code:

```
Object x = "x"; List y = (List)x;
```

Rectified code:

Object x = "x"; if(x instanceof List<?>){ List<?> y = (List<?>) x; }

Explanation:

Check type before casting; use generics.

V54 — Hardcoded path separator

Violated code:

```
String p = "C:\\temp\\file.txt";
```

Rectified code:

```
java.nio.file.Path p = java.nio.file.Paths.get(System.getProperty("java.io.tmpdir"), "file.txt");
```

Explanation:

Use `java.nio.file` for platform-independent paths.

V55 — String concatenation in loop

Violated code:

```
String big = ""; for(int i=0;i<100;i++){ big = big + i; }
```

Rectified code:

```
StringBuilder sb = new StringBuilder(256); for(int i=0;i<100;i++){ sb.append(i);} String big = sb.toString();
```

Explanation:

Use `StringBuilder` for repeated concatenation.

V56 — Same as V55

Violated code:

```
// see V55
```

Rectified code:

```
// see V55
```

Explanation:

Consolidated with V55.

V57 — Raw collection and mixed types

Violated code:

```
List items = new ArrayList(); items.add("one"); items.add(2);
```

Rectified code:

```
List<String> items = new ArrayList<>(); items.add("one");
```

Explanation:

Use generics to prevent heterogeneous lists.

V58 — Resource leak (FileInputStream not closed)

Violated code:

```
FileInputStream fis = new FileInputStream(path); byte[] buf = new byte[1024]; fis.read(buf);
```

Rectified code:

```
try(FileInputStream fis = new FileInputStream(DEFAULT_PATH)){ byte[] buf = new byte[1024]; int read = fis.read(buf); LOG.info("Read byte
```

Explanation:

Use `try-with-resources` and check read count.

V59 — Shadowed 'path' usage

Violated code:

```
FileInputStream fis = new FileInputStream(path);
```

Rectified code:

```
FileInputStream fis = new FileInputStream(DEFAULT_PATH);
```

Explanation:

Avoid shadowing; use clearly named constants.

V60 — Ignoring result of read()

Violated code:

```
fis.read(buf);
```

Rectified code:

```
int read = fis.read(buf); if(read == -1){ /* handle EOF */ }
```

Explanation:

Always check read count.

V61 — Stream not closed

Violated code:

```
// fis not closed
```

Rectified code:

```
// closed via try-with-resources (see V58)
```

Explanation:

Ensure closure via try-with-resources.

V62 — Empty catch on IO

Violated code:

```
catch(Exception ex){}
```

Rectified code:

```
catch(java.io.IOException ex){ LOG.log(java.util.logging.Level.WARNING, "IO failed", ex); }
```

Explanation:

Catch specific exceptions.

V63 — Printing internal mutable array

Violated code:

```
System.out.println(java.util.Arrays.toString(GLOBAL_DATA));
```

Rectified code:

```
System.out.println(java.util.Arrays.toString(getGlobalDataCopy()));
```

Explanation:

Expose only defensive copies when logging/returning.

V64 — Return internal mutable arrays

Violated code:

```
public static String[] getGlobalData(){ return GLOBAL_DATA; }
```

Rectified code:

```
public static String[] getGlobalData(){ return java.util.Arrays.copyOf(GLOBAL_DATA, GLOBAL_DATA.length); }
```

Explanation:

Return defensive copies (see also V121).

V65 — Suspicious equals/hashCode behavior

Violated code:

```
System.out.println("Equals? "+ we1.equals(we2));
```

Rectified code:

```
// Fixed by correcting equals/hashCode in class (see V129–V132)
```

Explanation:

Fix at the class level; equals must be deterministic and match hashCode.

V66 — Busy-wait loop

Violated code:

```
long start = System.currentTimeMillis(); while(System.currentTimeMillis()-start < 5){}
```

Rectified code:

```
try{ Thread.sleep(5);}catch(InterruptedException ie){ Thread.currentThread().interrupt(); }
```

Explanation:

Avoid spin-wait; sleep or schedule.

V67 — Insecure Random for security tokens

Violated code:

```
java.util.Random r = new java.util.Random(); String token = "" + r.nextInt();
```

Rectified code:

```
java.security.SecureRandom sr = new java.security.SecureRandom(); byte[] bytes = new byte[16]; sr.nextBytes(bytes); String token = java.util.Base64.encodeBase64(bytes);
```

Explanation:

Use SecureRandom for tokens.

V68 — Predictable token generation

Violated code:

```
String token = "" + r.nextInt();
```

Rectified code:

```
// see V67
```

Explanation:

Consolidated with V67.

V69 — Duplicated magic numbers

Violated code:

```
int size = 42; int limit = 42;
```

Rectified code:

```
private static final int DEFAULT_SIZE = 42; int size = DEFAULT_SIZE; int limit = DEFAULT_SIZE;
```

Explanation:

Use named constants.

V70 — Double-checked locking without volatile

Violated code:

// BadSingleton without volatile

Rectified code:

// Use holder idiom or volatile (see V133)

Explanation:

Use holder idiom or volatile with correct DCL.

V71 — SQL Injection via concatenation

Violated code:

```
DB.runQuery("SELECT * FROM users WHERE name = " + userInput + "');
```

Rectified code:

```
DbService.runQuery("SELECT * FROM users WHERE name = ?", java.util.List.of(userInput));
```

Explanation:

Parameterize queries.

V72 — Command injection via shell

Violated code:

```
Runtime.getRuntime().exec("sh -c echo " + userInput);
```

Rectified code:

```
ProcessBuilder pb = new ProcessBuilder("echo", userInput); try{ Process p=pb.start(); int exit=p.waitFor(); if(exit!=0) LOG.warning("echo failed"); }
```

Explanation:

Avoid shell; pass args as separate parameters; validate inputs.

V73 — Swallowed IOException

Violated code:

```
catch (java.io.IOException io) { }
```

Rectified code:

```
catch (java.io.IOException io) { LOG.log(java.util.logging.Level.WARNING, "Process IO failed", io); }
```

Explanation:

Log and handle exceptions.

V74 — InterruptedException swallowed/not reset

Violated code:

```
catch (InterruptedException ie){ }
```

Rectified code:

```
catch (InterruptedException ie){ Thread.currentThread().interrupt(); LOG.log(java.util.logging.Level.FINE, "Interrupted", ie); }
```

Explanation:

Always restore interrupt status.

V75 — Legacy Date API misuse

Violated code:

```
java.util.Date date = new java.util.Date(); date.setYear(120);
```

Rectified code:

```
java.time.LocalDate date = java.time.LocalDate.of(2020,1,1);
```

Explanation:

Use `java.time` (JSR-310).

V76 — Cloneable misused (shallow clone)

Violated code:

```
dto.clone();
```

Rectified code:

```
DTO copy = new DTO(dto);
```

Explanation:

Prefer explicit copy constructors/factories.

V77 — Serializable without serialVersionUID

Violated code:

```
class DTO implements java.io.Serializable { }
```

Rectified code:

```
class DTO implements java.io.Serializable { private static final long serialVersionUID = 1L; }
```

Explanation:

Define `serialVersionUID` for stable serialization.

V78 — Unused results

Violated code:

```
"HELLO".toLowerCase();
```

Rectified code:

```
String lower = "HELLO".toLowerCase(); LOG.fine(lower);
```

Explanation:

Use result or remove call.

V79 — Magic sleep with no retry/backoff

Violated code:

```
try{ Thread.sleep(5);}catch(Exception ignored){}
```

Rectified code:

```
// Use a retry policy with backoff/jitter (conceptual)
```

Explanation:

Introduce retry strategies (exponential backoff, jitter) instead of fixed sleeps.

V80 — Long if-else chain

Violated code:

```
if(n==1){...} else if(n==2){...} else if(n==3){...} else {...}
```

Rectified code:

```
java.util.Map<Integer,String> map = java.util.Map.of(1,"1",2,"2",3,"3",4,"4",5,"5"); printMessage(map.getOrDefault(n, "other"));
```

Explanation:

Replace with lookup/strategy.

V81 — Shadowing field with local

Violated code:

```
String path = "/tmp/data.txt";
```


Rectified code:

```
String dataPath = "/tmp/data.txt";
```

Explanation:

Avoid shadowing; use distinct names.

V82 — Ignored AutoCloseable (no TWR)

Violated code:

```
BufferedReader br = new BufferedReader(new FileReader(path));
```

Rectified code:

```
try(BufferedReader br = new BufferedReader(new FileReader(dataPath))){ String line; while((line=br.readLine())!=null){ /* process */ } } catch
```

Explanation:

Use try-with-resources for deterministic closure.

V83 — Logging sensitive data to stdout

Violated code:

```
System.out.println("LINE: " + line);
```

Rectified code:

```
LOG.fine() -> "Read line (len): " + line.length());
```

Explanation:

Avoid logging raw sensitive data; prefer metadata.

V84 — Swallowed IOException

Violated code:

```
catch(java.io.IOException ioe){}
```

Rectified code:

```
catch(java.io.IOException ioe){ LOG.log(java.util.logging.Level.WARNING, "Read failed", ioe); }
```

Explanation:

Always log/handle IO errors.

V85 — Insecure SSL context (trust-all)

Violated code:

```
ctx.init(null, new TrustManager[]{new InsecureTrustAllManager()}, new java.security.SecureRandom());
```

Rectified code:

```
ctx.init(null, null, null);
```

Explanation:

Do not trust-all; use default trust managers or vetted trust store.

V86 — Swallowed SSL exception

Violated code:

```
catch (Exception ex){ }
```

Rectified code:

```
catch (Exception ex){ LOG.log(java.util.logging.Level.WARNING, "SSL init failed", ex); }
```

Explanation:

Log and handle failures.

V87 — Thread-unsafe mutation of shared list

Violated code:

```
sharedList.add("x");
```

Rectified code:

```
SHARED_LIST.add("x");
```

Explanation:

Use thread-safe collection wrapper.

V88 — Misleading names (I vs O)

Violated code:

```
int I = 1; int O = 2;
```

Rectified code:

```
int one = 1; int two = 2;
```

Explanation:

Avoid ambiguous identifiers.

V89 — Long parameter list

Violated code:

```
Util.doEverything(1,2,3,4,5,6,7,8,9,10,"x","y",true,false);
```

Rectified code:

```
DoEverythingParams p = new DoEverythingParams.Builder().a(1).b(2).c(3).d(4).e(5).f(6).g(7).h(8).i(9).j(10).x("x").y("y").z1(true).z2(false).bu
```

Explanation:

Use parameter object/builder.

V90 — Duplicated code region

Violated code:

```
for(int i=0;i<3;i++){ counter++; }
```

Rectified code:

```
incrementCounter(COUNTER, 3);
```

Explanation:

Extract and reuse methods.

V91 — Direct floating-point equality

Violated code:

```
if(piApprox == 3.14){ /*...*/ }
```

Rectified code:

```
final double EPS = 1e-9; if (Math.abs(piApprox - 3.14) < EPS) { /*...*/ }
```

Explanation:

Compare with tolerance (epsilon).

V92 — Unused synchronized block

Violated code:

```
synchronized (BadApp.class){ }
```

Rectified code:

// removed

Explanation:

Remove no-op synchronization.

V93 — Legacy collections (Hashtable/Vector)

Violated code:

```
Hashtable ht = new Hashtable(); Vector v = new Vector();
```

Rectified code:

```
Map<String,String> map = new HashMap<>(); List<String> v = new ArrayList<>();
```

Explanation:

Prefer modern collections unless legacy semantics needed.

V94 — Calling System.gc()

Violated code:

```
System.gc();
```

Rectified code:

// removed

Explanation:

Let JVM manage GC.

V95 — Unnecessary finalizer usage

Violated code:

```
// finalize() used for cleanup
```

Rectified code:

```
// Use AutoCloseable with TWR (see V142–V145)
```

Explanation:

Avoid finalizers; use try-with-resources.

V96 — Deep nesting & early returns

Violated code:

```
if(userInput != null){ if(userInput.length()>0){ if(!userInput.isEmpty()){ if(userInput.trim().length()>0){ /*...*/ } else { return; } } else { return; } } e
```

Rectified code:

```
if(userInput == null || userInput.trim().isEmpty()){ return; } LOG.info("userInput seems non-empty");
```

Explanation:

Flatten conditions; fail fast.

V97 — Dead code after returns

Violated code:

```
System.out.println("This might not execute.");
```

Rectified code:

```
// remove or relocate
```

Explanation:

Avoid unreachable code.

V98 — Exception as control flow

Violated code:

```
try { throw new RuntimeException("flow"); } catch (RuntimeException ex){}
```

Rectified code:

```
// Use normal conditionals for flow
```

Explanation:

Use exceptions only for exceptional cases.

V99 — Unused method call result

Violated code:

```
Util.returnMutableArray();
```

Rectified code:

```
String[] data = Util.returnImmutableList().toArray(new String[0]);
```

Explanation:

Use the result and avoid exposing mutability.

V100 — Using System.err directly

Violated code:

```
System.err.println("done with bad things");
```

Rectified code:

```
LOG.severe("done");
```

Explanation:

Use logger at appropriate level.

V101 — Excessive comments and clutter

Violated code:

```
// many noisy comments
```

Rectified code:

```
// concise comments; self-documenting code
```

Explanation:

Maintain clarity; remove redundant comments.

V102 — Too many responsibilities in main

Violated code:

```
// monolithic main
```

Rectified code:

```
// decomposed into App/AppRunner/Services (see V10)
```

Explanation:

Split responsibilities.

V103 — No unit tests

Violated code:

```
// no tests
```

Rectified code:

```
// add unit tests (JUnit/Mockito) for DB, IO, utils
```

Explanation:

Introduce tests to prevent regressions.

V104 — No input validation/sanitation

Violated code:

```
// direct use of user input
```

Rectified code:

```
// validate length/charset/whitelist; parameterize DB queries
```

Explanation:

Always validate and sanitize inputs early.

V105 — Integer overflow/impractical loop

Violated code:

```
for(int i=0;i<Integer.MAX_VALUE; i++){ if(i==5) break; }
```

Rectified code:

```
for(int i=0;i<6;i++){ /*...*/ }
```

Explanation:

Avoid extreme bounds; use explicit limits.

V106 — Useless boolean expression

Violated code:

```
if(true || false){}
```

Rectified code:

```
// removed
```

Explanation:

Eliminate tautologies.

V107 — Duplicate literal strings

Violated code:

```
System.out.println("duplicate"); System.out.println("duplicate");
```

Rectified code:

```
LOG.info("duplicate");
```

Explanation:

Avoid duplication; consolidate.

V108 — DB security best practices ignored

Violated code:

```
// concatenated SQL; root creds
```

Rectified code:

```
// use PreparedStatement and least-privilege (see V156–V163)
```

Explanation:

Parameterize queries; avoid root; restrict privileges.

V109 — Race condition iterating sharedList

Violated code:

```
for(Object obj: sharedList){ System.out.println(obj); }
```

Rectified code:

```
synchronized(SHARED_LIST){ for(Object obj: SHARED_LIST){ LOG.info(String.valueOf(obj)); } }
```

Explanation:

Synchronize when iterating synchronized lists (or use concurrent collections).

V110 — Ignoring character encodings

Violated code:

```
byte[] bytes = "text".getBytes();
```

Rectified code:

```
byte[] bytes = "text".getBytes(java.nio.charset.StandardCharsets.UTF_8);
```

Explanation:

Always specify charset.

V111 — Optional omission/misuse

Violated code:

```
// potential null usage
```

Rectified code:

```
java.util.Optional<String> maybe = getMaybe(); maybe.ifPresent(v -> LOG.info(v));
```

Explanation:

Use Optional to model absence.

V112 — Logging before level check

Violated code:

```
LOG.fine("expensive: " + compute());
```

Rectified code:

```
LOG.fine(() -> "expensive: " + compute());
```

Explanation:

Use lambda supplier so computation happens only if enabled.

V113 — Relying on HashMap iteration order

Violated code:

```
for(Object k: config.keySet()){ System.out.println(k); }
```

Rectified code:

```
CONFIG.entrySet().stream().sorted(java.util.Map.Entry.comparingByKey()).forEach(e -> LOG.info(e.getKey()));
```

Explanation:

HashMap order is unspecified; sort if order matters.

V114 — Law of Demeter violation

Violated code:

```
new DeepA().getB().getC().getD().action();
```

Rectified code:

```
deepAction(); // façade/mediator
```

Explanation:

Use façade to avoid deep chains.

V115 — Potential memory leak via static cache

Violated code:

```
Util.cache.put("k","v");
```

Rectified code:

```
private static final java.util.Map<String,String> CACHE =  
    java.util.Collections.synchronizedMap(new java.util.LinkedHashMap<>(16,0.75f,true){  
        @Override protected boolean removeEldestEntry(java.util.Map.Entry<String,String> e){ return size() > 1000; }  
    });
```

Explanation:

Bound caches or use weak refs to prevent unbounded growth.

V116 — Thread.sleep used for timing

Violated code:

```
try{ Thread.sleep(1);}catch(Exception ex){}
```

Rectified code:

```
java.util.concurrent.ScheduledExecutorService ses = java.util.concurrent.Executors.newSingleThreadScheduledExecutor(); ses.schedule(()
```

Explanation:

Prefer ScheduledExecutorService for timers.

V117 — Calling finalize() explicitly

Violated code:

```
LeakyResource lr = new LeakyResource(); lr.finalize();
```

Rectified code:

```
try(LeakyResourceFix lr = new LeakyResourceFix()){ /* use */ }
```

Explanation:

Never call finalize(); use AutoCloseable.

V118 — System.exit in library-like code

Violated code:

```
System.exit(0);
```

Rectified code:

```
// Throw exception or return error; only top-level app may exit
```

Explanation:

Libraries should not terminate JVM.

V119 — Default-locale formatting

Violated code:

```
String.format("Pi=%.2f", piApprox);
```

Rectified code:

```
String.format(java.util.Locale.ROOT, "Pi=%.2f", piApprox);
```

Explanation:

Specify locale for deterministic formatting.

V120 — Inefficient contains on List

Violated code:

```
for(int i=0;i<10;i++){ if(items.contains(i)){ /* ... */ } }
```

Rectified code:

```
java.util.Set<Integer> set = new java.util.HashSet<>(items); for(int i=0;i<10;i++){ if(set.contains(i)){ /*...*/ } }
```

Explanation:

Use a Set for O(1) membership.

V121 — Method returns internal array

Violated code:

```
public static String[] getGlobalData(){ return GLOBAL_DATA; }
```

Rectified code:

```
public static String[] getGlobalData(){ return java.util.Arrays.copyOf(GLOBAL_DATA, GLOBAL_DATA.length); }
```

Explanation:

Return defensive copy.

V122 — Risky method allows injection

Violated code:

```
st = conn.createStatement(); st.executeUpdate(dangerousSql);
```

Rectified code:

```
try(java.sql.PreparedStatement ps = conn.prepareStatement("UPDATE users SET active=? WHERE name=?")){ ps.setBoolean(1,true); ps.
```

Explanation:

Use PreparedStatement with parameters.

V123 — Resources not closed in all paths

Violated code:

```
Connection conn = null; Statement st = null;
```

Rectified code:

```
try(Connection conn = java.sql.DriverManager.getConnection(url,user,pass)){ /* ... */ }
```

Explanation:

Use try-with-resources for Connection/Statement/ResultSet.

V124 — Hardcoded JDBC URL/user/password

Violated code:

```
DriverManager.getConnection("jdbc:mysql://localhost:3306/test","root","root");
```

Rectified code:

```
DriverManager.getConnection(System.getenv("DB_URL"), System.getenv("DB_USER"), System.getenv("DB_PASS"));
```

Explanation:

Read DB config from environment/config files.

V125 — Statement instead of PreparedStatement

Violated code:

```
conn.createStatement();
```

Rectified code:

```
conn.prepareStatement(sql);
```

Explanation:

PreparedStatement prevents injection and may improve performance.

V126 — Executing user-controlled SQL

Violated code:

```
st.executeUpdate(dangerousSql);
```

Rectified code:

```
// Use parameterized PreparedStatement (see V122)
```

Explanation:

Never execute raw user SQL.

V127 — Swallowing exception in DB call

Violated code:

```
catch(Exception ex){}
```

Rectified code:

```
catch(java.sql.SQLException ex){ LOG.log(java.util.logging.Level.SEVERE, "DB failure", ex); }
```

Explanation:

Log SQLExceptions with context.

V128 — Connection not closed in finally

Violated code:

```
// conn not closed
```

Rectified code:

```
// Closed by try-with-resources in V123
```

Explanation:

Always close connection via TWR.

V129 — equals without hashCode; weird equals

Violated code:

```
class WeirdEquals { int id; public boolean equals(Object o){ return o instanceof WeirdEquals && ((WeirdEquals)o).id==this.id && new java.u
```

Rectified code:

```
final class EntityId { private final int id; EntityId(int id){ this.id=id;} @Override public boolean equals(Object o){ if(this==o) return true; if(!(o ins
```

Explanation:

Deterministic equals and matching hashCode; encapsulate fields.

V130 — Package-private field

Violated code:

```
int id;
```

Rectified code:

```
private final int id;
```

Explanation:

Encapsulate state.

V131 — Randomness in equals

Violated code:

```
... && new Random().nextBoolean()
```

Rectified code:

Deterministic comparison only

Explanation:

equals must be reflexive/symmetric/transitive/consistent.

V132 — Missing hashCode

Violated code:

```
// no hashCode
```

Rectified code:

```
@Override public int hashCode(){ return Integer.hashCode(id); }
```

Explanation:

Provide hashCode consistent with equals.

V133 — Non-thread-safe singleton (DCL without volatile)

Violated code:

```
private static BadSingleton instance; // no volatile
```

Rectified code:

```
final class GoodSingleton { private GoodSingleton(){} private static class Holder{ private static final GoodSingleton INSTANCE = new GoodSingleton(); }}
```

Explanation:

Use holder idiom or volatile with correct DCL.

V134 — Exposing 'this' in constructor

Violated code:

```
BadApp.sharedList.add(this);
```

Rectified code:

```
// Avoid leaking this from constructor; publish after fully constructed
```

Explanation:

Do not publish partially constructed objects.

V135 — Runnable with side effects and no handling

Violated code:

```
class BadRunnable implements Runnable { public void run(){ /* ... */ } }
```

Rectified code:

```
final class GoodRunnable implements Runnable { @Override public void run(){ try{ /* work */ } catch(RuntimeException e){ LOG.log(java.util.logging.Level.SEVERE, e.getMessage(), e); } }
```

Explanation:

Handle exceptions; avoid hidden side effects.

V136 — Unbounded thread creation (commented)

Violated code:

```
// new Thread(this).start();
```

Rectified code:

```
// Use ExecutorService instead of recursive thread creation
```

Explanation:

Manage threads via executors.

V137 — Swallowing exceptions in runnable

Violated code:

```
catch (Exception e){}
```

Rectified code:

```
catch (Exception e){ LOG.log(java.util.logging.Level.SEVERE, "Worker failed", e); }
```

Explanation:

Log/propagate worker failures.

V138 — DTO implements Cloneable/Serializable (anti-pattern)

Violated code:

```
class DTO implements Cloneable, java.io.Serializable { }
```

Rectified code:

```
class DTO implements java.io.Serializable { private static final long serialVersionUID = 1L; }
```

Explanation:

Avoid Cloneable; define serialVersionUID if serializable.

V139 — Raw type field in DTO

Violated code:

```
public List data = new ArrayList();
```

Rectified code:

```
private final List<String> data = new ArrayList<>();
```

Explanation:

Use generics and encapsulation.

V140 — Shallow clone risk

Violated code:

```
return super.clone();
```

Rectified code:

```
// Use deep copy constructor: new DTO(other)
```

Explanation:

Implement deep copy explicitly.

V141 — Missing serialVersionUID

Violated code:

```
// none
```

Rectified code:

```
private static final long serialVersionUID = 1L;
```

Explanation:

Provide serialVersionUID.

V142 — finalize() with resource leak

Violated code:

```
protected void finalize(){ if(in!=null) in.close(); }
```

Rectified code:

```
final class LeakyResourceFix implements AutoCloseable { private final java.io.InputStream in; LeakyResourceFix() throws java.io.IOException {
```

Explanation:

Use `AutoCloseable` and `try-with-resources`; avoid `finalize()`.

V143 — Not closing `FileInputStream` in ctor

Violated code:

```
in = new FileInputStream("leaky.txt");
```

Rectified code:

```
// Open in ctor and close in close() (see V142)
```

Explanation:

Ensure deterministic closure.

V144 — Swallowing exceptions in ctor

Violated code:

```
catch (Exception e){}
```

Rectified code:

```
catch (java.io.IOException e){ throw new RuntimeException("Failed to open resource", e); }
```

Explanation:

Wrap/rethrow with context; do not swallow.

V145 — Using `finalize` for cleanup

Violated code:

```
@Override protected void finalize() { /* ... */ }
```

Rectified code:

```
// No finalize; rely on AutoCloseable and TWR
```

Explanation:

Avoid `finalize`; it is unreliable/deprecated.

V146 — Utility class non-private ctor & public static mutable

Violated code:

```
class Util { public static Map cache = new HashMap(); Util(){} }
```

Rectified code:

```
final class Util { private Util(){ throw new AssertionError(); } private static final java.util.Map<String,String> CACHE = new java.util.concurrent.ConcurrentHashMap<>(); }
```

Explanation:

Make ctor private; avoid exposing mutable statics.

V147 — Raw type + public static mutable cache

Violated code:

```
public static Map cache = new HashMap();
```

Rectified code:

```
private static final java.util.Map<String,String> CACHE = new java.util.concurrent.ConcurrentHashMap<>();
```

Explanation:

Encapsulate and use generics.

V148 — Utility constructor should be private

Violated code:

Util(){}

Rectified code:

```
private Util(){ throw new AssertionError(); }
```

Explanation:

Prevent instantiation of utility class.

V149 — Long parameter list with magic numbers

Violated code:

```
int doEverything(int a,int b,int c,int d,int e,int f,int g,int h,int i,int j,String x,String y,boolean z1,boolean z2)
```

Rectified code:

```
int doEverything(DoEverythingParams p)
```

Explanation:

Replace with parameter object with named fields.

V150 — Long expression with magic numbers

Violated code:

```
result += a+b+c+d+e;
```

Rectified code:

```
result += p.sumFirstGroup();
```

Explanation:

Encapsulate operations; avoid magic numbers.

V151 — Duplicate logic

Violated code:

```
result += f+g+h+i+j;
```

Rectified code:

```
result += p.sumSecondGroup();
```

Explanation:

Refactor duplicate logic.

V152 — String compare with ==

Violated code:

```
if ("x"==x){ result++; }
```

Rectified code:

```
if ("x".equals(x)) { result++; }
```

Explanation:

Use equals for strings.

V153 — Law of Demeter deep chain

Violated code:

```
new DeepA().getB().getC().getD().action();
```

Rectified code:

```
doDeepAction(); // façade method
```

Explanation:

Provide façade; reduce coupling.

V154 — Returns internal mutable array

Violated code:

```
public static String[] returnMutableArray(){ return BadApp.GLOBAL_DATA; }
```

Rectified code:

```
public static java.util.List<String> returnImmutableList(){ return java.util.List.of(BadApp.getGlobalData()); }
```

Explanation:

Return immutable/copy, not internal array.

V155 — System.out in deep call

Violated code:

```
void action(){ System.out.println("deep"); }
```

Rectified code:

```
void action(){ LOG.fine("deep"); }
```

Explanation:

Use logger, not stdout.

V156 — DB helper unsafe (Statement, no TWR)

Violated code:

```
Statement s = c.createStatement(); ResultSet rs = s.executeQuery(sql);
```

Rectified code:

```
try(PreparedStatement ps = c.prepareStatement(sql)){ /* set params */ try(ResultSet rs = ps.executeQuery()){ while(rs.next()){ LOG.fine() -
```

Explanation:

Use PreparedStatement and try-with-resources.

V157 — Hardcoded DB credentials

Violated code:

```
DriverManager.getConnection("jdbc:mysql://localhost/test","root","root");
```

Rectified code:

```
DriverManager.getConnection(System.getenv("DB_URL"), System.getenv("DB_USER"), System.getenv("DB_PASS"));
```

Explanation:

Move credentials to environment/secret store.

V158 — No PreparedStatement

Violated code:

```
c.createStatement();
```

Rectified code:

```
c.prepareStatement(sql);
```

Explanation:

Parameterize queries.

V159 — Potential injection in executeQuery(sql)

Violated code:

```
s.executeQuery(sql);
```

Rectified code:

```
ps.executeQuery();
```

Explanation:

Never pass concatenated SQL.

V160 — System.out for DB results

Violated code:

```
System.out.println(rs.getString(1));
```

Rectified code:

```
LOG.fine() -> rs.getString(1);
```

Explanation:

Use logger; avoid printing sensitive DB data.

V161 — Swallowing DB exceptions

Violated code:

```
catch(Exception ex){}
```

Rectified code:

```
catch(java.sql.SQLException ex){ LOG.log(java.util.logging.Level.SEVERE, "DB error", ex); }
```

Explanation:

Log SQLExceptions.

V162 — Incorrect close order & leaks

Violated code:

```
// partial resource closing; connection not closed
```

Rectified code:

```
// Use nested try-with-resources for rs, ps, conn (see V156)
```

Explanation:

Close resources correctly via TWR.

V163 — Connection not closed

Violated code:

```
// connection left open
```

Rectified code:

```
// closed by try-with-resources on Connection
```

Explanation:

Always close Connection.

V164 — Insecure trust manager (trust-all)

Violated code:

```
class InsecureTrustAllManager implements X509TrustManager { /* trust all */ }
```

Rectified code:

```
// Remove trust-all; use default TrustManager or proper trust store
```

Explanation:

Never disable TLS verification.

V165 — getAcceptedIssuers returns null

Violated code:

```
public java.security.cert.X509Certificate[] getAcceptedIssuers(){ return null; }
```

Rectified code:

```
public java.security.cert.X509Certificate[] getAcceptedIssuers(){ return new java.security.cert.X509Certificate[0]; }
```

Explanation:

Return empty array, not null, if implementing.

V166 — Trust-all client check

Violated code:

```
public void checkClientTrusted(java.security.cert.X509Certificate[] chain, String authType){}
```

Rectified code:

```
// Perform proper validation or delegate to default manager
```

Explanation:

Implement validation; do not accept all certificates.

V167 — Trust-all server check

Violated code:

```
public void checkServerTrusted(java.security.cert.X509Certificate[] chain, String authType){}
```

Rectified code:

```
// Perform proper validation or delegate to default manager
```

Explanation:

Implement validation; do not accept all certificates.